Practical No. 1: Fundamentals of R – Basic Operations and Vectors

Aim: To study basic R syntax, variables, arithmetic operations, vectors, and the mean function.

Program / R Code

```
# Program 1: Basic arithmetic operations and vectors

a <- 20
b <- 10
addition <- a + b
 subtraction <- a - b
multiplication <- a * b
division <- a / b
print(addition)
print(subtraction)
print(multiplication)
print(division)

marks <- c(65, 70, 75, 80, 85)
print(marks)
print(mean(marks))
```

OUTPUT

```
[1] 30
[1] 10
[1] 200
[1] 2
[1] 65 70 75 80 85
[1] 75
```

Practical No. 2: Fundamentals of R – Data Frame and Descriptive Statistics

Aim: To create a data frame and calculate the mean, median, minimum, maximum, and standard deviation of marks.

Program / R Code

```
Name <- c("Amit", "Neha", "Rahul", "Priya", "Kiran")
Marks <- c(75, 82, 68, 90, 78)
Age <- c(20, 21, 20, 21, 20)

student <- data.frame(Name, Marks, Age)
print(student)

print(mean(student$Marks))
print(median(student$Marks))
print(min(student$Marks))
print(max(student$Marks))
print(sd(student$Marks))
```

OUTPUT

```
  Name Marks Age
1 Amit 75 20
2 Neha 82 21
3 Rahul 68 20
4 Priya 90 21
5 Kiran 78 20
[1] 78.6
[1] 78
[1] 68
[1] 90
[1] 8.173127
```

Practical No. 3: Graphs Using R – Line Graph

Aim: To represent monthly sales graphically using a line graph.

Program / R Code

```
month <- c("Jan", "Feb", "Mar", "Apr", "May")
sales <- c(20, 25, 30, 28, 35)

month <- factor(month, levels = c("Jan", "Feb", "Mar", "Apr", "May"))

plot(month, sales,
type = "o",
main = "Monthly Sales",
xlab = "Month",
ylab = "Sales")
```

OUTPUT



Practical No. 4: Graphs Using R – Bar Graph

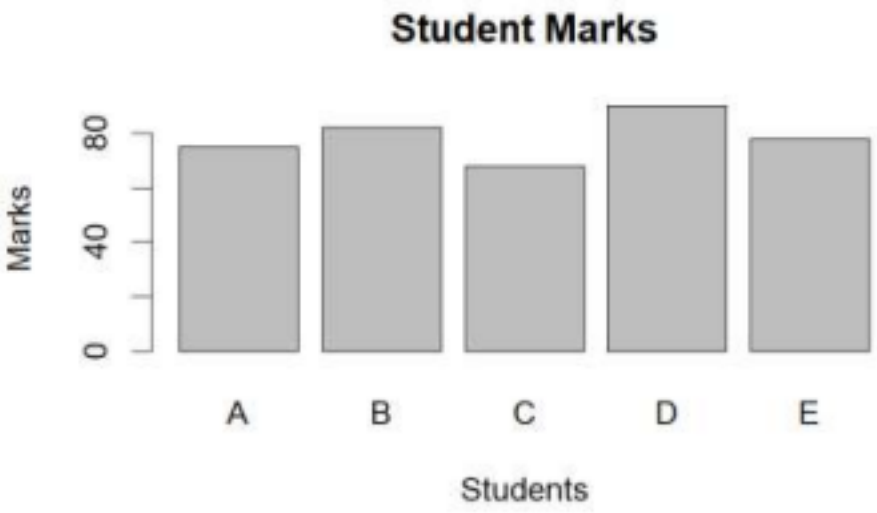
Aim: To represent categorical numerical data using a bar graph.

.Program / R Code

```
students <- c("A", "B", "C", "D", "E")
marks <- c(75, 82, 68, 90, 78)

barplot(marks,
 names.arg = students,
 main = "Student Marks",
 xlab = "Students",
 ylab = "Marks")
```

OUTPUT



Practical No. 5: Graphs Using R – Histogram

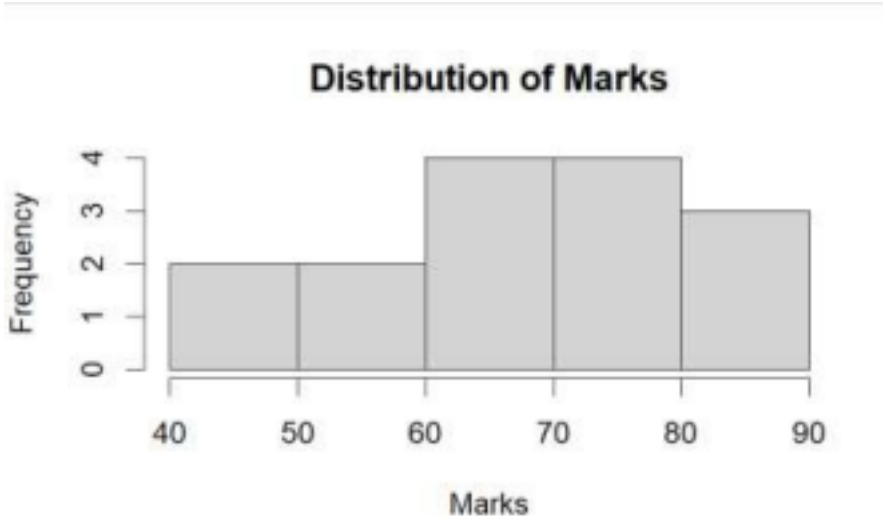
Aim: To study the frequency distribution of numerical marks using a histogram.

Program / R Code

```
# Practical 5: Histogram
marks <- c(45, 50, 55, 60, 62, 65, 68,
 70, 72, 75, 78, 80, 82, 85, 90)

hist(marks,
 main = "Distribution of Marks",
 xlab = "Marks",
 ylab = "Frequency")
```

OUTPUT



Practical No. 6: Diagrams Using R – Pie Chart

Aim: To represent parts of a whole using a pie chart.

Program / R Code

```
# Practical 6: Pie Chart
expenses <- c(30, 25, 20, 15, 10)
categories <- c("Food", "Rent", "Travel", "Education", "Others")

pie(expenses,
 labels = categories,
 main = "Monthly Expenses")
```

OUTPUT



Practical No. 7: Diagrams Using R – Box Plot

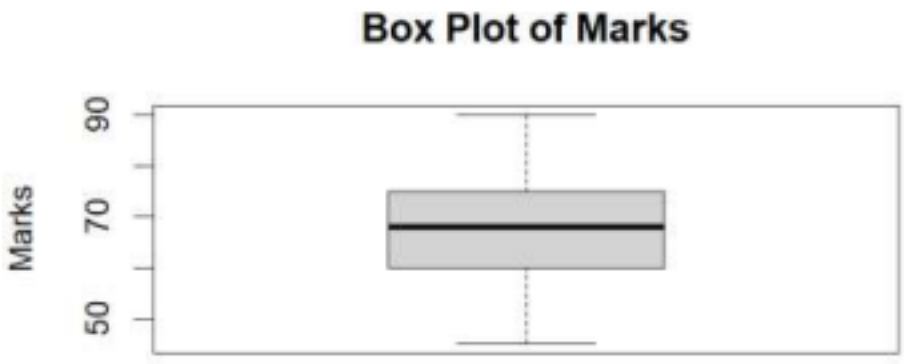
Aim: To represent the distribution, median, spread, and possible outliers of numerical marks using a box plot.

Program / R Code

```
# Practical 7: Box Plot
marks <- c(45, 50, 55, 60, 62, 65,
 68, 70, 72, 75, 80, 85, 90)

boxplot(marks,
 main = "Box Plot of Marks",
 ylab = "Marks")
```

OUTPUT



Practical No. 8: Simple Linear Regression

Question: Write an R program to fit a simple linear regression model to study the relationship between advertising expenditure and sales.

Aim: To fit a simple linear regression model and obtain the regression coefficients.

Theory: Simple linear regression studies the linear relationship between one dependent variable Y and one independent variable X. In R, lm(Y ~ X) fits the linear model. The fitted equation is of the form $\hat{Y} = b0 + b1X$.

Program / R Code

```
# Practical 8: Simple Linear Regression
advertising <- c(10, 20, 30, 40, 50)
sales <- c(16, 24, 37, 43, 56)

model <- lm(sales ~ advertising)

print(model)
summary(model)
```

OUTPUT

```
Call:
lm(formula = sales ~ advertising)

Coefficients:
(Intercept)  advertising
       5.50         0.99

> summary(model)

Call:
lm(formula = sales ~ advertising)

Residuals:
   1    2    3    4    5
 0.6 -1.3  1.8 -2.1  1.0

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.50000    1.98074   2.777 0.069178 .
advertising  0.99000    0.05972  16.577 0.000478 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.889 on 3 degrees of freedom
Multiple R-squared:  0.9892,    Adjusted R-squared:  0.9856
F-statistic: 274.8 on 1 and 3 DF,  p-value: 0.0004779
```

Practical No. 9: Simple Linear Regression with Scatter Plot and Regression Line

Aim: To visualize the relationship between advertising expenditure and sales and display the fitted regression line.

Program / R Code

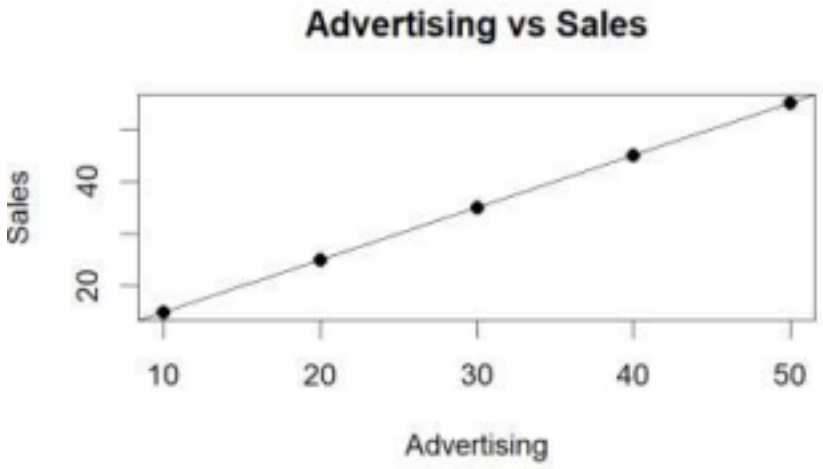
```
# Practical 9: Scatter Plot with Regression Line
advertising <- c(10, 20, 30, 40, 50)
sales <- c(15, 25, 35, 45, 55)

model <- lm(sales ~ advertising)

plot(advertising, sales,
 main = "Advertising vs Sales",
 xlab = "Advertising",
 ylab = "Sales",
 pch = 19)

abline(model)
```

OUTPUT



Practical No. 10: Weighted Least Squares Regression

Aim: To fit a regression model in which observations are assigned different weights.

Program / R Code

```
# Practical 10: Weighted Least Squares Regression
x <- c(1, 2, 3, 4, 5, 6)
y <- c(2, 4, 5, 8, 10, 12)

weights <- c(1, 1, 2, 2, 3, 3)

model <- lm(y ~ x, weights = weights)

print(model)
summary(model)
```

OUTPUT

```
Call:
lm(formula = y ~ x, weights = weights)

Coefficients:
(Intercept)            x
    -0.4944       2.0787

> summary(model)

Call:
lm(formula = y ~ x, weights = weights)

Weighted Residuals:
      1       2       3       4       5       6
 0.41573  0.33708 -1.04874  0.25424  0.17515  0.03892

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.4944     0.4979  -0.993    0.377
x             2.0787     0.1118  18.593 4.92e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.6089 on 4 degrees of freedom
Multiple R-squared:  0.9886,    Adjusted R-squared:  0.9857
F-statistic: 345.7 on 1 and 4 DF,  p-value: 4.925e-05
```

Practical No. 11: Multiple Linear Regression

Aim: To fit and interpret a multiple linear regression model with two independent variables. **Program / R**

Code

```
# Practical 11: Multiple Linear Regression
study_hours <- c(2, 3, 4, 5, 6, 7)
attendance <- c(60, 65, 70, 80, 90, 95)
marks <- c(50, 55, 65, 72, 85, 90)

model <- lm(marks ~ study_hours + attendance)

print(model)
summary(model)
```

OUTPUT

```
Call:
lm(formula = marks ~ study_hours + attendance)

Coefficients:
(Intercept)  study_hours   attendance
      2.216        3.486        0.673

> summary(model)

Call:
lm(formula = marks ~ study_hours + attendance)

Residuals:
      1       2       3       4       5       6
 0.4324 -1.4189  1.7297 -1.4865  1.2973 -0.5541

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)    2.2162    18.3393   0.121    0.911
study_hours    3.4865     3.1598   1.103    0.350
attendance     0.6730     0.4215   1.597    0.209

Residual standard error: 1.769 on 3 degrees of freedom
Multiple R-squared:  0.9926,    Adjusted R-squared:  0.9877
F-statistic: 202.5 on 2 and 3 DF,  p-value: 0.0006304
```

Practical No. 12: Prediction Using Multiple Linear Regression

Aim: To use a fitted multiple linear regression model for prediction.

Program / R Code

```
# Practical 12: Prediction Using Multiple Linear Regression
study_hours <- c(2, 3, 4, 5, 6, 7)
attendance <- c(60, 65, 70, 80, 90, 95)
marks <- c(50, 55, 65, 72, 85, 90)

model <- lm(marks ~ study_hours + attendance)

new_data <- data.frame(
 study_hours = 5,
 attendance = 85
)

prediction <- predict(model, newdata = new_data)

print(prediction)
```

OUTPUT

```
> print(prediction)
       1
76.85135
```

Practical No. 13: Ridge Regression Using R

Aim: To implement Ridge Regression and obtain coefficient estimates for a range of penalty values.

Program / R Code

```
# Practical 13: Ridge Regression
# Run once if glmnet is not installed:
# install.packages("glmnet")

library(glmnet)

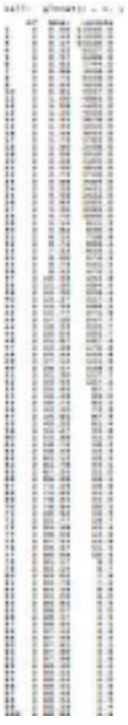
x <- matrix(c(
 2, 60,
 3, 65,
 4, 70,
 5, 80,
 6, 90
), ncol = 2, byrow = TRUE)

y <- c(50, 55, 65, 72, 85)

model <- glmnet(x, y, alpha = 0)

print(model)
```

OUTPUT



Practical No. 14: Ridge Regression with Cross-Validation

Aim: To select a suitable penalty parameter lambda for Ridge Regression using cross-validation.

Program / R Code

```
# Practical 14: Ridge Regression with Cross-Validation
# Run once if glmnet is not installed:
# install.packages("glmnet")

library(glmnet)

x <- matrix(c(
 2, 60,
 3, 65,
 4, 70,
 5, 80,
 6, 90
), ncol = 2, byrow = TRUE)

y <- c(50, 55, 65, 72, 85)

set.seed(123)

cv_model <- cv.glmnet(x, y, alpha = 0)

print(cv_model$lambda.min)
print(cv_model$lambda.1se)

plot(cv_model)
```

OUTPUT

```
[1] [lambda.min produced by R]
[1] [lambda.1se produced by R]
```

